

Nivya

Complete Software Requirements Specification

Version 1.0 — Final Consolidated Specification

Personal Family Safety • Device Health • Activity Status • Location • Usage • Convocation

Document Status	FINAL
Version	1.0
Primary Platform	Android first
Backend	Java + Spring Boot
Database	MySQL
Web Dashboard	React
Realtime	WebSocket + Redis
Notifications	FCM

Final consolidated SRS containing the original Nivya requirements and all subsequent UI, role, pairing, dashboard, activity, history, child-view, and Convocation updates.

Project name and app name are always written as **Nivya**. The visual wordmark may be styled, but the product name is not changed.

1. Document Purpose

This Software Requirements Specification defines the complete functional and technical requirements for Nivya. It consolidates the project requirements discussed for the Parent and Child mobile experiences, the persistent connection model, device status and health information, permitted activity status, parent history, and the independent Convocation feature.

Nivya is intended as a personal family-safety and device-status system. Parent and Child accounts are separate, the relationship is established through an explicit connection process, and the selected role determines exactly which interface and options are rendered.

2. Product Vision

Nivya should feel like a clean, modern phone-status and device-health application rather than a complicated control panel. The opening dashboard is a simple summary. Specific information is opened through separate Options/Side Menu items.

The central experience is: **Connect once** → **remain linked** → **synchronize permitted device information** → **present role-specific information clearly**.

3. Roles and Visibility

Area	Parent	Child
Account	Parent account	Child account
Dashboard	Clean summary	Clean summary
Detailed activity	Available	Not rendered
History	Available	Not rendered
App usage details	Available	Not rendered as parent-side detail
Location detail	Available	Current location/status only
Battery	Available	Available
Screen Time	Available	Available
Network	Available	Available
Network Quality	Available	Available
Device Health	Available	Available
Alerts	Available	Parent alerts/notifications
Clean Up	Parent can view/manage supported device status	Available Child has cleanup option
Convocation	Available	Available
Family/Device management	Available	Not rendered
Pairing controls	Available	Not rendered after connection
Parent settings	Available	Not rendered

Important UI rule: the Child interface must not show labels such as “Parent Only”, “Common”, or “Parent Access”. Instead, the application renders only the options valid for the selected role. Parent-only controls are simply absent from the Child interface.

4. Application Launch and Login Flow

```
App Open
↓
Login Screen
↓
Successful Authentication
↓
Select Role: Parent / Child
↓
Connection Screen
↓
Enter opposite device/account connection code
↓
Automatic connection
↓
Role-specific Dashboard
```

The role-selection page is the next page after login. Selecting Parent or Child changes the application experience. The wrong-role interface is not merely hidden behind a label; it should not be rendered into the selected role's navigation model.

5. Parent-Child Connection

5.1 Connection Codes

Immediately after role selection and before the main application is entered, the user receives a connection screen. Each side displays its own connection code and provides a field for entering the opposite person's code.

```
Parent phone
Your Connection Code: NV-4821-KP90
Enter Child's Code: [_____]

Child phone
Your Connection Code: NV-7315-QA26
Enter Parent's Code: [_____]
```

Entering the opposite device/account's valid code causes the backend to validate the relationship and automatically establish the connection.

5.2 Automatic Connection

Once a valid opposite code is entered and the pairing conditions are satisfied, both accounts/devices become linked. No separate repeated pairing step should be required for normal use.

5.3 Persistent Relationship

The family relationship remains active until explicitly revoked, the account/device is removed, or security credentials are invalidated. A normal reboot, app restart, temporary network loss, or temporary backend unavailability must not destroy the relationship.

5.4 Reboot / Offline

```
Phone restart
→ Nivya initializes
→ secure device identity restored
→ session validation
→ connection restored
→ synchronization resumes

Phone powered off / unreachable
→ Parent sees Offline / Last Seen
→ device reconnects when available
→ synchronization resumes
```

6. Parent Dashboard

The Parent dashboard is intentionally a clean summary and should not contain every detail at once.

Nivya

Child Device
ONLINE

Battery 78%
Location Home
Screen Time 2h 18m
Network 5G
Network Strong
Device Health Good
Alerts 1

Last Sync 22:16

Detailed information is opened through the Side Menu / Options.

6.1 Parent Side Menu

- Dashboard
- Live Activity
- History
- App Usage
- Calls / communication status where supported and authorized
- Location
- Device Health
- Alerts
- Convocation
- Family / Devices
- Settings

7. Child Dashboard

The Child dashboard is deliberately limited to the Child-facing device-care information defined for Nivya.

Nivya

Battery 62%
Screen Time 1h 35m
Network 4G
Network Quality Excellent
Location Near Home
Device Health Good
Alerts 1

The Child dashboard may also surface temporary Parent alerts/messages in the normal alert area. It must not become a Parent-control dashboard.

7.1 Child Side Menu

- Dashboard
- Battery
- Screen Time
- Network
- Location
- Device Health
- Clean Up

- Alerts
- Convocation
- Settings

No Parent-only controls, Family/Device management, detailed Live Activity, or detailed History options are rendered in the Child menu.

8. Device Status and Health

8.1 Battery

- Current battery percentage
- Charging / not charging state
- Full state where available
- Battery health information where the operating system exposes it
- Battery temperature where safely available
- Battery trend/history for Parent
- Configurable low-battery alerts

8.2 Network

- Connected / disconnected
- Wi-Fi / mobile-data state where available
- Network type such as 4G/5G/Wi-Fi where available
- Signal level where available
- Internet availability
- Last synchronization time

8.3 Network Quality — Child

The Child view must include a simple Network Quality indication:

- Excellent — strong and stable connection
- Good — stable connection
- Weak — limited connection
- Unavailable — no usable network information

If the platform does not expose a reliable value, Nivya must show Unavailable rather than fabricate a result.

8.4 Device Health

- Overall device health status
- Storage availability where available
- Application/device status
- Permission status relevant to Nivya
- Health detail screen
- Clear unavailable states

9. Child Device Care Features

9.1 Screen Time

Child can view their current/summary Screen Time. Parent can view the corresponding usage information in the Parent experience.

9.2 Current Location Status

Child can see a simple current-location status such as “Near Home”, “Location Available”, or “Location Unavailable”. Detailed location information belongs to the Parent experience according to the enabled sharing and platform capabilities.

9.3 Clean Up

The Child menu contains a Clean Up option for supported temporary files/cache/device cleanup operations. The feature should explain what will be removed before the operation and should not delete personal files without an explicit action.

10. Live Activity

Parent has a separate Live Activity option. It is not mixed into the dashboard.

The activity layer represents app/device activity at a status level, without capturing private communication content. Where the operating system and application integration provide the information, examples include:

- WhatsApp — Chatting with: Contact Name
- Files — Viewing: document.pdf
- Browser — Browsing
- Phone — In call with: Contact Name, where supported
- Video application — Watching
- Document application — Viewing document

When a specific in-app detail cannot be reliably obtained from the operating system or available integration, Nivya must display a less specific state such as “Active in app” or “Information unavailable” rather than inventing a value.

11. Parent-Only History

History is a dedicated Parent option. It is not shown as a mixed dashboard summary and is not rendered in the Child navigation.

History records the available activity timeline using time, application, broad activity/status, available contact/document label, and duration.

```
10:05 PM
WhatsApp
Chatting with Arun
Duration: 04 min

10:18 PM
Files
Viewing: report.pdf
Duration: 03 min

10:42 PM
Phone
In call with Mom
Duration: 12 min
```

The exact fields stored depend on what the operating system and authorized integration can provide. Private message bodies, call audio, keystrokes, screenshots, and similar content capture are outside this SRS.

12. Location

- Parent can view the child's current/last permitted location.

- Parent can view location history according to retention settings.
- Location must show last update time and accuracy/availability state.
- Child sees only the Child-facing current location/status screen.
- Location sharing requires the required device permission.
- If permission is denied or revoked, the UI shows Permission Required / Disabled / Unavailable.

Parent:
 Aarav's Location
 Updated: 2 minutes ago
 Accuracy: Approximate

Child:
 Location
 Near Home
 Updated: 2 minutes ago

13. App Usage

Parent can open a dedicated App Usage section to view application usage summaries, daily totals, weekly totals, and application/category breakdowns where the platform provides the required usage data.

Today's Usage
 YouTube 42 min
 Instagram 28 min
 Chrome 35 min
 WhatsApp 26 min
 Spotify 18 min
 Other 45 min

Total: 2h 14m

The Child can see Screen Time as a device-care summary. Detailed Parent-side usage history is not rendered as a Child navigation option.

14. Communication Status

Where the operating system and authorized permissions make communication metadata available, Nivya may show high-level status such as call count, incoming/outgoing status, duration, or a current call contact label.

Nivya does not capture call audio or private message content. Communication status is treated as metadata and is subject to platform availability and permission state.

15. Alerts and Parent Guidance

15.1 Device Alerts

- Low battery
- Device offline
- Device online/reconnected
- Location stale/unavailable
- Safety event
- Security event

15.2 Parent Guidance

Parent may send a short guidance message/alert such as "Check your battery status" or "Drink water". The Child receives the configured generic notification behavior defined for the product. This communication is separate from detailed Parent-side history.

16. Convocation — Independent Feature

Convocation is an independent feature. Changes to Convocation must not modify the behavior of the Battery, Network, Location, Usage, Device Health, History, Live Activity, or other Nivya modules.

Convocation is visible to both Parent and Child, but the two views have intentionally different behavior.

16.1 Parent Convocation

Parent can open Convocation and view the complete retained messages received from the Child. Parent can send messages to the Child.

```
Convocation — Parent

Parent:
Charge your phone. 10:12 PM

Parent:
Drink some water. 10:15 PM

Child:
I reached home. 10:20 PM

Child:
I am studying. 10:35 PM
```

Parent-side Child message history remains available according to the product's retention policy.

16.2 Child Convocation — Menu

Convocation must appear in the Child Side Menu. The Child Convocation screen has a top-right three-dot Options icon.

```
Child Side Menu

Dashboard
Battery
Screen Time
Network
Location
Device Health
Clean Up
Alerts
Convocation
Settings
```

16.3 Child Convocation — Default

When the Child opens Convocation, the content area is an empty text-space/notepad-style view. Parent messages are not directly displayed by default.

```
Convocation ■

[ empty ]

No messages to show right now.

Write something... [ > ]
```

16.4 Options Menu and Single On/Off Control

The top-right Options icon opens the Convocation Options menu. The Turn On/Off behavior is represented by one single toggle, not separate Turn On and Turn Off menu items.

```
■ Options

[ Turn On ■ ]

Clear Now
How it works?
```

When enabled, the same control becomes:


```
■ Options

[ Turn On ● ]

Clear Now
How it works?
```

The exact visual label may be “Turn On” with a toggle, or a state-aware single toggle control. The important requirement is that there is only one On/Off control inside the Options menu.

16.5 Parent → Child Notification

Whenever Parent sends a Convocation message, the Child's notification text must always be:

```
Check your battery status
```

The notification must not expose the Parent's actual message. Tapping the notification opens Nivya normally and must not deep-link directly into the Convocation chat.

16.6 Unread Parent Messages

If Parent sends two or more messages before the Child views them, the unread messages accumulate as the current unread set. When the Child turns the single toggle ON, all currently unread Parent messages in that set can be viewed.

```
Parent sends:
1. Charge your phone.
2. Drink some water.
3. Check your battery.

Child turns ON:
→ all three currently unread messages are shown.
```

This is intentionally different from a “latest message only” rule for unread messages. The Child can view multiple Parent messages that were unread when the Child enabled the view.

16.7 Seen Status

After the Child views the Parent message set, the **Seen** status is shown only on the Parent side. The Child side must not display “Seen”.

```
Parent side:

Parent:
Charge your phone. Seen

Parent:
Drink some water. Seen

Parent:
Check your battery. Seen
```

The Child simply sees the message content while it is visible; the read-state indicator is a Parent-side state.

16.8 Two-Minute Visibility / Auto Hide

The Child-side Parent-message viewing state automatically turns OFF after 2 minutes. The displayed Parent messages disappear from the Child view and the Convocation returns to its empty state.

```
Turn ON
↓
Unread Parent messages visible
↓
2 minutes
↓
Automatically OFF
↓
Messages hidden
↓
Empty Convocation
```

16.9 One-Hour Viewed Message Expiry

Once the Child has viewed the Parent messages, those viewed messages expire from Child-side visibility after 1 hour. This does not delete the Parent's retained history.

The two-minute automatic OFF controls the temporary viewing mode. The one-hour rule controls the lifetime of the viewed-message visibility record on the Child side. The Parent-side retained history is unaffected.

16.10 New Parent Messages After Viewing

If Parent sends a new message after the Child has viewed the previous message set, the new message becomes the new unread set. The Child can turn the single control ON again to view the new unread message(s). Previously viewed messages do not return to the Child view.

```
Old messages
→ Child viewed
→ Child-side visibility expires

Parent sends:
"Sleep early today."

Next time Child turns ON:
→ "Sleep early today." is available
```

16.11 Child → Parent

The Child can type and send a Convocation message. Immediately after sending, that Child-side message disappears from the Child interface. The Child does not receive a persistent personal message history.

The Parent receives and retains the Child message in Parent-side Convocation history.

```
Child:
"I reached home."
↓
Send
↓
Child screen returns to empty
↓
Parent history:
"I reached home." ← retained
```

16.12 Convocation State Model

State	Child View	Parent View
OFF / default	Empty	Full retained history
ON with unread set	Unread Parent messages visible	Messages visible + read state can update
Child viewed	Content may remain only under defined expiry rule	Seen label
2-minute auto-off	Empty / hidden	History unaffected
New Parent message	New unread set becomes available	Added to full history
Child sends message	Own message disappears immediately	Message retained in history

17. Notifications

- Parent receives configured device and safety notifications.
- Child receives Parent guidance notifications according to the Convocation notification rule.
- Convocation Parent notification text shown to Child is always "Check your battery status".
- Notification tap opens Nivya normally and does not deep-link directly into Convocation.
- Notification content must not expose the actual Parent message.

18. Family and Devices

Parent can manage the family relationship and linked devices. Child does not receive Parent family-management controls in the Child navigation.

- View linked child/device
- View device status
- Add/remove supported family members
- Revoke device relationship
- Retire old devices
- Manage alert rules

19. Android Application Architecture

```
nivya-android/  
■■■■ app/  
■■■■ core/  
■ ■■■■ network/  
■ ■■■■ security/  
■ ■■■■ permissions/  
■ ■■■■ database/  
■ ■■■■ utils/  
■■■■ data/  
■ ■■■■ local/  
■ ■■■■ remote/  
■ ■■■■ repository/  
■ ■■■■ model/  
■■■■ domain/  
■ ■■■■ model/  
■ ■■■■ usecase/  
■■■■ ui/  
■ ■■■■ onboarding/  
■ ■■■■ auth/  
■ ■■■■ role/  
■ ■■■■ pairing/  
■ ■■■■ dashboard/  
■ ■■■■ battery/  
■ ■■■■ network/  
■ ■■■■ location/  
■ ■■■■ usage/  
■ ■■■■ device/  
■ ■■■■ alerts/  
■ ■■■■ convocation/  
■ ■■■■ settings/  
■■■■ services/  
■■■■ sync/  
■■■■ location/  
■■■■ notifications/
```

Recommended Android technologies: Kotlin, Jetpack Compose, Room, WorkManager, Coroutines, Retrofit/OkHttp, DataStore, FCM, and approved platform APIs for location and usage information.

20. React Web Dashboard

```
nivya-web/  
■■■■ src/  
■■■■ components/  
■ ■■■■ BatteryCard/  
■ ■■■■ NetworkCard/  
■ ■■■■ LocationCard/  
■ ■■■■ UsageCard/  
■ ■■■■ ActivityCard/  
■ ■■■■ HistoryCard/  
■ ■■■■ AlertCard/  
■ ■■■■ DeviceCard/  
■■■■ pages/  
■ ■■■■ Login/
```

- ■■■ RoleSelection/
- ■■■ Dashboard/
- ■■■ Family/
- ■■■ Device/
- ■■■ LiveActivity/
- ■■■ History/
- ■■■ Location/
- ■■■ Usage/
- ■■■ Alerts/
- ■■■ Settings/
- services/
- store/
- hooks/
- layouts/
- utils/

21. Spring Boot Backend Architecture

```
com.nivya
■■■ auth/
■■■ user/
■■■ family/
■■■ child/
■■■ pairing/
■■■ device/
■■■ battery/
■■■ network/
■■■ location/
■■■ usage/
■■■ activity/
■■■ history/
■■■ communication/
■■■ alerts/
■■■ notification/
■■■ convocation/
■■■ consent/
■■■ privacy/
■■■ audit/
■■■ safezone/
■■■ common/
```

Backend responsibilities include authentication, authorization, family relationships, device registration, telemetry, location, usage summaries, activity/history records, alerts, notifications, Convocation storage, privacy state, and audit logging.

22. MySQL Database Design

Table	Purpose
users	Parent/Child account records
roles	Role definitions
families	Family records
family_members	User-to-family relationship
devices	Registered child devices
device_sessions	Device session credentials
pairing_requests	Connection-code/pairing state
pairing_tokens	Short-lived pairing tokens
consents	Terms/privacy/sharing acceptance
privacy_settings	Sharing configuration

Table	Purpose
device_status	Online/offline and status telemetry
battery_status	Battery records
network_status	Network records
locations	Permitted location records
location_history	Historical location records
usage_summary	Daily/weekly usage aggregates
usage_apps	Application usage records
activity_events	Available activity-status events
history_events	Parent-visible historical activity
alerts	Generated alert records
alert_rules	Threshold/configuration rules
notifications	Notification records
convocation_messages	Convocation message records
convocation_views	Child-view/read-state/expiry metadata
audit_logs	Security and account audit events
safe_zones	Optional safe-zone definitions
safe_zone_events	Optional zone entry/exit events
refresh_tokens	Refresh token state

23. Key Database Structures

23.1 users

```

id
uuid
name
email
phone
password_hash
role
status
created_at
updated_at
last_login_at

```

23.2 devices

```

id
device_uuid
user_id
family_id
device_name
platform
os_version
app_version
push_token
status
last_seen_at
created_at
updated_at

```

23.3 convocation_messages

```
id
family_id
sender_user_id
receiver_user_id
message_text
created_at
viewed_at
child_visibility_expires_at
status
```

Parent-side retention and Child-side visibility must be represented separately. A Child-side expiry must not erase the Parent's retained history.

23.4 convocation_views

```
id
message_id
child_id
viewed_at
seen_recorded_at
child_visible_until
status
```

The read-state is surfaced as “Seen” only in the Parent UI.

24. REST API

Authentication

```
POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/verify
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
POST /api/v1/auth/change-password
POST /api/v1/auth/forgot-password
```

Role / Pairing

```
POST /api/v1/pairing/create
POST /api/v1/pairing/connect
GET /api/v1/pairing/status
POST /api/v1/pairing/revoke
```

Devices

```
POST /api/v1/devices/register
GET /api/v1/devices/{deviceId}
POST /api/v1/devices/{deviceId}/heartbeat
POST /api/v1/devices/{deviceId}/telemetry
GET /api/v1/devices/{deviceId}/status
```

Location / Usage / Activity

```
POST /api/v1/location/update
GET /api/v1/location/current/{deviceId}
GET /api/v1/location/history/{deviceId}

POST /api/v1/usage/sync
GET /api/v1/usage/today/{deviceId}
GET /api/v1/usage/week/{deviceId}

GET /api/v1/activity/live/{deviceId}
GET /api/v1/history/{deviceId}
```

Convocation

```
GET /api/v1/convocation/messages
POST /api/v1/convocation/messages
POST /api/v1/convocation/view
```

```
POST /api/v1/convocation/toggle
POST /api/v1/convocation/clear
```

25. Real-Time Communication

WebSocket may be used for near-real-time status updates.

```
DEVICE_ONLINE
DEVICE_OFFLINE
BATTERY_UPDATED
NETWORK_UPDATED
LOCATION_UPDATED
USAGE_SYNCED
ACTIVITY_UPDATED
ALERT_CREATED
PAIRING_CONNECTED
CONVOCAATION_MESSAGE_AVAILABLE
CONVOCAATION_VIEWED
```

FCM may be used for push notifications. The notification payload for a Parent Convocation message must not contain the actual message text when delivered to the Child.

26. Security

- HTTPS for all network communication.
- JWT access tokens with short lifetime.
- Refresh-token rotation and revocation.
- BCrypt or Argon2 password hashing.
- Role-based access control.
- Server-side family/device authorization.
- Pairing codes are random, short-lived, and rate-limited.
- Audit logs for security-sensitive actions.
- Input validation and centralized exception handling.
- Rate limiting on login and pairing endpoints.
- Never log passwords, tokens, or unnecessary sensitive content.
- WebSocket connections must authenticate and authorize the family/device scope.

Access-control principle: a Parent request must be checked against the authenticated Parent's actual family and linked child/device relationship. A user must not gain access merely by changing an ID in a URL.

27. Consent and Privacy

Parent and Child must explicitly accept the relevant terms and sharing explanations before the family/device relationship is activated.

- Terms of Service
- Privacy Policy
- Sharing/Monitoring disclosure
- Location permission explanation
- Usage-access explanation
- Relevant platform permission explanations

The application should maintain document version and acceptance timestamp. If a future legal/privacy document version materially changes, the product may require renewed acceptance.

28. Data Retention

Data	Suggested Initial Retention
Current device status	Short-lived/current state
Detailed location history	30 days
Detailed usage	90 days
Parent activity history	Configurable product policy
Convocation Parent history	Configurable product policy
Child-side viewed-message visibility	1 hour after viewing
Temporary pairing token	Until expiration
Refresh token	Until expiry/revocation

Retention values are product defaults and can be changed later without altering the core architecture.

29. Offline and Synchronization

```
No internet
↓
Permitted data stored temporarily on device
↓
Network returns
↓
Retry with backoff
↓
Server validates
↓
Sync succeeds
↓
Temporary synced queue cleared
```

The persistent family relationship remains intact during offline periods. If the phone is powered off, no live telemetry can be transmitted until the device is available again.

30. Alert Engine

```
IF battery <= configured threshold
THEN LOW_BATTERY

IF heartbeat absent for configured duration
THEN DEVICE_OFFLINE

IF location update becomes stale
THEN LOCATION_STALE

IF configured safety event occurs
THEN SAFETY_ALERT
```

Parent can configure supported thresholds. Child sees only the Child-facing alert information intended by the product.

31. Optional Safe Zones and SOS

Future/optional modules may include Safe Zones and SOS.

- Safe Zone: Home, College, Tuition, Hostel, etc.
- Entry/exit alerts based on permitted location.
- SOS can send an emergency event, permitted current location, battery, and timestamp.
- These modules remain separate from Convocation.

32. UI Design Requirements

- App name always displayed as Nivya.
- Modern, clean, friendly visual language.
- Dashboard shows summaries only.
- Specific information is opened through separate options.
- Parent and Child navigation are role-specific.
- Child menu must not show Parent-only or Common labels.
- Unavailable information must be represented honestly.
- Convocation has its own independent UI.

33. Development Project Structure

```
nivya/  
■■■ backend/  
■■■ android/  
■■■ web/  
■■■ database/  
■■■ docs/  
■■■ docker/  
■■■ README.md
```

A monorepo is recommended for the first version so the backend, Android client, web dashboard, database migrations, architecture documents, and deployment configuration remain versioned together.

34. Development Order

1. Project setup and repository
2. MySQL schema and migrations
3. Spring Boot backend
4. JWT authentication
5. Parent and Child roles
6. Role selection UI
7. Connection-code pairing
8. Persistent device identity
9. Heartbeat and online/offline
10. Battery telemetry
11. Network telemetry and quality
12. Parent dashboard
13. Child dashboard
14. Location
15. Usage / Screen Time
16. Live Activity status
17. Parent-only History
18. Alerts and FCM
19. Convocation

20. Privacy/consent
21. Audit logging
22. Unlink and deletion
23. Security testing
24. Real-device testing
25. Production deployment

35. Testing Requirements

35.1 Functional

- Parent registration/login
- Child registration/login
- Role selection
- Connection-code generation
- Opposite-code entry
- Automatic connection
- Persistent relationship
- Reboot/reconnect
- Battery and charging
- Network and quality
- Location
- Usage
- Live Activity
- History
- Alerts
- Convocation message behavior
- Seen state on Parent only
- Two-minute auto-hide
- One-hour Child-side expiry
- Child message disappearance
- Parent retained history

35.2 Security

- Unauthorized family access
- Wrong device ID
- Expired pairing code
- Repeated pairing attempts
- Expired/revoked tokens
- WebSocket authorization
- Rate limiting

- Broken access control
- Input validation

35.3 Device

- Normal startup
- Reboot
- No network
- Wi-Fi/mobile transition
- Battery saver
- Permission denied/revoked
- Location disabled
- Usage access disabled
- App update
- Backend outage

36. Acceptance Criteria

Requirement	Acceptance
Login	Parent and Child can authenticate successfully.
Role selection	Parent/Child selection determines the rendered experience.
Pairing	Entering the opposite valid connection code connects both sides automatically.
Persistence	Normal reboot does not require pairing again.
Parent dashboard	Summary-only dashboard is displayed.
Child dashboard	Only Child-defined device-care information is rendered.
Child menu	No Parent-only/Common labels or Parent controls appear.
Network quality	Excellent/Good/Weak/Unavailable states work.
History	Parent can open a separate detailed history.
Live Activity	Parent can open status-level activity where available.
Convocation menu	Convocation appears in both Parent and Child menus.
Convocation options	Child uses a single On/Off toggle inside the Options menu.
Unread messages	All currently unread Parent messages are visible when Child turns ON.
Seen	Seen appears only on Parent side after Child views messages.
Auto-hide	Child viewing mode automatically turns OFF after 2 minutes.
Viewed expiry	Viewed Parent messages expire from Child-side visibility after 1 hour.
Child message	Child's sent message disappears immediately from Child view but remains in Parent history.
Notification	Parent Convocation messages notify Child only as "Check your battery status".
Notification tap	Tap opens Nivya normally, not directly into Convocation.
New messages	After viewed messages, a new Parent message becomes the new unread set.

37. Final End-to-End Experience

```
APP OPEN
↓
LOGIN
↓
SELECT ROLE
■■■ PARENT
■■■ CHILD
↓
CONNECTION SCREEN
↓
ENTER OPPOSITE CONNECTION CODE
↓
AUTOMATIC CONNECTION
↓
ROLE-SPECIFIC DASHBOARD
↓
SIDE MENU / OPTIONS
■■■ Parent: detailed Parent features
■■■ Child: Child device-care features
↓
PERSISTENT LINK
↓
SYNC / STATUS / ALERTS
```

38. Final Convocation Experience

```
PARENT
↓
Convocation
↓
Full Child-message history
↓
Parent sends message
↓
Child notification:
"Check your battery status"
↓
Child opens Nivya normally
↓
Child opens Convocation
↓
Options (■)
↓
Single ON/OFF toggle
↓
ON
↓
All currently unread Parent messages appear
↓
Child views them
↓
Seen is recorded/displayed ONLY for Parent
↓
2-minute viewing mode ends / turns OFF
↓
Child view becomes empty again
↓
Viewed messages expire from Child-side visibility after 1 hour
↓
Parent sends new message
↓
New unread set becomes available next time Child turns ON

CHILD → PARENT
↓
Child writes message
↓
Send
```

↓
Child-side message disappears immediately
↓
Parent receives/retains message in Convocation history

39. Final Product Definition

Nivya is a role-aware, persistent family device-safety application. Parent and Child authenticate separately, connect using a one-time opposite-device connection code, and remain linked until the relationship is revoked. The Parent experience provides the detailed Nivya information defined for the linked device, including separate Live Activity and History views. The Child experience provides only its own device-care options and the independent Convocation feature.

The dashboard remains a clean summary. Specific details are opened through separate options. The Child navigation does not expose Parent controls. Convocation remains independent from all other Nivya modules and follows the final message visibility, notification, toggle, Seen, expiry, and history rules specified in this document.

Application name: Nivya.

40. Change Control

This document is Version 1.0 and represents the consolidated baseline after the requested updates. Future changes should be recorded as new revisions while preserving this Version 1.0 baseline for comparison.

When a future requirement changes one module, the change should not silently alter unrelated modules. In particular, Convocation is an independent feature and changes to it should not change the Battery, Network, Location, Usage, Device Health, History, Live Activity, or other module requirements unless explicitly requested.

Nivya

Version 1.0 — Mandatory Final Updates & Operational Specification

Consolidation note: This section is appended to the previously generated complete Version 1.0 SRS to close the six implementation gaps identified during final review. All existing requirements remain in force.

41. Android Special Permission Fallback UX

Nivya must distinguish normal runtime permissions from Android special-access permissions. Capabilities such as Android Usage Access (`PACKAGE_USAGE_STATS`) and `NotificationListenerService` access cannot be granted through a normal in-app runtime permission popup.

41.1 Usage Access

Before opening system settings, Nivya must show an educational explanation. It then launches the Android Usage Access settings using the appropriate platform Intent (for example, `Settings.ACTION_USAGE_ACCESS_SETTINGS`).

```
Nivya → Permission explanation → Allow Usage Access
→ Android system settings → user enables Nivya
→ return to Nivya → verify permission
→ Granted: continue/sync
→ Not granted: Required / Try Again
```

41.2 Notification Listener

If `NotificationListenerService` is required for an implemented feature, Nivya must explain the need, open the Android notification-listener settings, return to Nivya, verify the service state, and provide Retry/Continue behavior.

```
Nivya → Notification Access explanation
→ Android notification-listener settings
→ user enables Nivya → return
→ verify service → Enabled / Required / Try Again
```

41.3 Fallback States

- Required — capability cannot operate until enabled.
- Disabled — previously enabled but later revoked.
- Unavailable — device/Android version does not expose the capability.
- Checking — Nivya is verifying the state after returning from Settings.

Nivya must never fabricate usage, activity, notification, or device information when a required permission is unavailable.

42. Convocation Timer Enforcement When App Is Killed

The Child Convocation 2-minute viewing rule must not rely solely on an in-memory timer. The backend records the viewing start and authoritative visibility expiry. Android also uses lifecycle-aware scheduling such as `WorkManager` and/or `AlarmManager`, subject to Android scheduling constraints, to clear local state promptly.

```
Turn ON → server records view_started_at
→ visibility_expires_at = view_started_at + 2 minutes
→ Child displays unread set
→ app background/killed/recreated
→ on resume + scheduled check
→ expired: toggle OFF + clear Child-visible messages
```

Server-side expiry is authoritative. Local scheduling is an additional UX enforcement mechanism, preventing a terminated app from keeping Parent messages visible indefinitely.

43. Telemetry Scaling Strategy

Battery, network, location, heartbeat, usage, and activity telemetry can generate large volumes. The production design must separate current state from historical raw telemetry and must use aggregation/archival.

- Keep latest device state in current-state tables for fast dashboard reads.
- Batch telemetry inserts rather than one transaction per event.
- Index `device_id`, `family_id`, timestamp and event type as appropriate.
- Keep raw high-frequency telemetry for a bounded period.
- Aggregate older telemetry into hourly/daily summaries where needed.

- Archive or delete raw data after configured retention.
- Use pagination for History and location/usage screens.
- Use date/device partitioning if production volume warrants it.

0-7 days: raw operational telemetry
 7+ days: aggregate to hourly/daily summaries
 after configured retention: archive/delete raw telemetry
 current-state tables: retain latest device state

The exact retention period is configurable; the architecture must support the strategy from the first production release.

44. Distribution and Google Play Compliance

Nivya Version 1.0 is specified for personal/private use and is not required to be published on Google Play. Therefore, Play Store publication is not a prerequisite for the private build.

If the application is later distributed publicly through Google Play or another store, that release must undergo a fresh policy review against the rules current at that time, including permissions, user-data disclosures, prominent disclosures, family/child requirements where applicable, and restrictions on device monitoring or sensitive-data access.

- Request only permissions needed by implemented features.
- Explain monitoring/sharing capabilities before enabling them.
- Maintain clear Terms and Privacy documentation.
- Use supported Android platform APIs and approved alternatives.
- Treat public-store policy review as a release gate.

45. Error Tracking and Operational Monitoring

Production must include crash/error reporting while avoiding unnecessary private data in diagnostics.

- Android: Firebase Crashlytics or Sentry.
- React Web Dashboard: Sentry or equivalent.
- Spring Boot: Sentry, OpenTelemetry-compatible monitoring, or equivalent.
- Monitor API errors, authentication failures, pairing failures, WebSocket reconnects, telemetry sync failures, database latency, Redis health, and background sync failures.

Crash and operational telemetry must not contain passwords, access tokens, private message bodies, call audio, or unnecessary personal information.

46. Updates and Remote Configuration

Nivya must define an update path for Android, web, and backend components.

- Private Android builds may use the selected private distribution mechanism.
- If Play is adopted later, use the update mechanism applicable to that release and current Play requirements.
- Firebase Remote Config or an equivalent server configuration service may manage feature flags, alert thresholds, staged rollouts, and non-code messaging.
- Remote configuration must not bypass Android permissions or silently enable capabilities.
- Backend must be able to enforce a minimum supported client version for critical security updates.

47. React Web Dashboard Real-Time Behavior

The React Parent Web Dashboard must maintain a persistent authenticated WebSocket connection to receive real-time telemetry, activity, alerts, device-state, and Convocation updates without manual page refresh.

React Parent Dashboard
 → authenticated WebSocket
 → Spring Boot WebSocket layer


```
→ Redis event distribution
→ telemetry/activity/alert/Convocation event
→ React store/state update
→ UI changes without refresh
```

47.1 Reconnection and Recovery

- Authenticate the WebSocket after Parent login.
- Subscribe only to the authorized family/device scope.
- Reconnect automatically after temporary network loss using bounded backoff.
- After reconnect, request a current state snapshot to fill missed events.
- Close/re-authenticate connections when the Parent session expires or is revoked.

47.2 Example Events

- BATTERY_UPDATED
- NETWORK_UPDATED
- NETWORK_QUALITY_UPDATED
- LOCATION_UPDATED
- DEVICE_ONLINE / DEVICE_OFFLINE
- ACTIVITY_UPDATED
- ALERT_CREATED
- CONVOCAION_MESSAGE_AVAILABLE
- CONVOCAION_VIEWED / SEEN_UPDATED

48. Expanded Complete Project Structure

The Version 1.0 project structure is expanded below so the SRS defines implementation boundaries, not only top-level folders.

```
Nivya/
■■■ backend/
■ ■■■ pom.xml
■ ■■■ src/
■ ■■■ main/java/com/nivya/
■ ■ ■■■ auth/
■ ■ ■■■ user/
■ ■ ■■■ role/
■ ■ ■■■ pairing/
■ ■ ■■■ family/
■ ■ ■■■ device/
■ ■ ■■■ battery/
■ ■ ■■■ network/
■ ■ ■■■ location/
■ ■ ■■■ usage/
■ ■ ■■■ activity/
■ ■ ■■■ history/
■ ■ ■■■ communication/
■ ■ ■■■ alerts/
■ ■ ■■■ notification/
■ ■ ■■■ convocation/
■ ■ ■■■ consent/
■ ■ ■■■ privacy/
■ ■ ■■■ audit/
■ ■ ■■■ websocket/
■ ■ ■■■ security/
■ ■ ■■■ common/
■ ■■■ main/resources/
■ ■■■ application.yml
■ ■■■ application-dev.yml
■ ■■■ application-prod.yml
```

```
■ ■■■ db/migration/
■
■■■ android/
■ ■■■ app/
■ ■■■ core/
■ ■ ■■■ network/
■ ■ ■■■ security/
■ ■ ■■■ permissions/
■ ■ ■■■ database/
■ ■ ■■■ utils/
■ ■■■ data/
■ ■ ■■■ local/
■ ■ ■■■ remote/
■ ■ ■■■ repository/
■ ■ ■■■ model/
■ ■■■ domain/
■ ■ ■■■ model/
■ ■ ■■■ usecase/
■ ■■■ ui/
■ ■ ■■■ auth/
■ ■ ■■■ role/
■ ■ ■■■ pairing/
■ ■ ■■■ dashboard/
■ ■ ■■■ battery/
■ ■ ■■■ screen_time/
■ ■ ■■■ network/
■ ■ ■■■ location/
■ ■ ■■■ device_health/
■ ■ ■■■ cleanup/
■ ■ ■■■ alerts/
■ ■ ■■■ convocation/
■ ■ ■■■ settings/
■ ■■■ services/
■ ■ ■■■ sync/
■ ■ ■■■ telemetry/
■ ■ ■■■ location/
■ ■ ■■■ usage/
■ ■ ■■■ notification/
■ ■ ■■■ reconnect/
■ ■■■ permissions/
■ ■■■ location/
■ ■■■ usage_access/
■ ■■■ notification_listener/
■
■■■ web/
■ ■■■ package.json
■ ■■■ src/
■ ■■■ components/
■ ■■■ pages/
■ ■ ■■■ Login/
■ ■ ■■■ RoleSelection/
■ ■ ■■■ Dashboard/
■ ■ ■■■ LiveActivity/
■ ■ ■■■ History/
■ ■ ■■■ Location/
■ ■ ■■■ Usage/
■ ■ ■■■ Alerts/
■ ■ ■■■ Convocation/
■ ■ ■■■ Settings/
■ ■■■ layouts/
■ ■■■ services/
■ ■■■ websocket/
■ ■■■ store/
■ ■■■ hooks/
■ ■■■ routes/
■ ■■■ utils/
■
■■■ database/
■ ■■■ migrations/
■ ■■■ seeds/
■ ■■■ indexes/
■ ■■■ procedures/
```

```
■■■■ docker/
■ ■■■■ backend/
■ ■■■■ mysql/
■ ■■■■ redis/
■ ■■■■ nginx/
■■■■ docs/
■ ■■■■ SRS/
■ ■■■■ API/
■ ■■■■ Architecture/
■ ■■■■ Database/
■ ■■■■ UI/
■ ■■■■ Deployment/
■■■■ scripts/
■ ■■■■ dev/
■ ■■■■ build/
■ ■■■■ backup/
■■■■ .env.example
■■■■ docker-compose.yml
■■■■ README.md
■■■■ LICENSE
```

48.1 Responsibility Boundaries

Backend packages own business logic, APIs, persistence, security, and real-time delivery. Android owns role-specific UI, device telemetry, permitted usage/location collection, local state, permission UX, notifications, and resilient synchronization. React owns the Parent web dashboard and its authenticated real-time experience. Database migrations and indexes are version-controlled separately.

49. Environment and Deployment

```
Development:
Android emulator/physical device
React dev server
Spring Boot dev profile
MySQL
Redis
```

```
Private production:
Android APK/private distribution
React build + Nginx
Spring Boot service
MySQL + tested backups
Redis
TLS
FCM
Crash/error monitoring
```

- Secrets must use environment variables or a secure secret store.
- Never commit API keys, JWT secrets, database passwords, or FCM credentials.
- Development and production databases remain separate.
- Backups must be tested periodically.
- Logs must avoid unnecessary private data.

50. Final Version 1.0 Review Checklist

- All original requirements remain represented.
- Login → Role Selection → Connection Code → Role-specific Dashboard is defined.
- Opposite connection codes automatically establish the Parent/Child relationship.
- Relationship persists across normal restart/reboot.
- Dashboard is a clean summary; details are opened separately.
- Child navigation renders only Child-facing options; Parent controls are absent.

- Network Quality is present on the Child side.
- Parent Live Activity and Parent-only History remain separate.
- Convocation is independent from other modules.
- Child Convocation contains the top-right Options icon.
- Options contains one single On/Off toggle.
- All currently unread Parent messages can be viewed when the Child turns ON.
- Seen appears only on the Parent side after Child viewing.
- Parent notification to Child always reads "Check your battery status".
- Notification tap opens Nivya normally, not directly into Convocation.
- Child messages disappear from Child view immediately after sending but remain in Parent history.
- 2-minute Convocation viewing expiry is enforced beyond app process lifetime.
- Child-side viewed-message expiry is separate from Parent-side retention.
- Telemetry has an aggregation/archival strategy.
- Private-use distribution is documented; future public distribution requires policy review.
- Crash/error monitoring and update strategy are defined.
- React Web Dashboard consumes authenticated persistent WebSocket updates.
- Expanded project structure is defined.